



pyflashdb

Unified Database Interface for Python

MySQL

PostgreSQL

MongoDB

One API. Any database. Zero boilerplate.

v1.0.4 · MIT License · Python 3.8+

pypi.org/project/pyflashdb/



Table of Contents

1. What is pyflashdb?
2. Installation
3. Connecting to a Database
4. CRUD — Insert
5. CRUD — Read
6. CRUD — Update
7. CRUD — Delete
8. Filter Syntax
9. Schema Operations
10. Trigger Hooks
11. Transactions
12. Raw Queries
13. MongoDB-Specific Features
14. Error Handling
15. Verifying Your Setup
16. Feature Matrix
17. Project Structure
18. Changelog

1. What is pyflashdb?

pyflashdb wraps MySQL, PostgreSQL, and MongoDB behind one clean Python API. You learn the methods once and they work on every database — no driver differences, no SQL dialect switching, no separate query builders.

The same line works on all three databases:

```
flash.add("users", {"name": " Komal ", "age": 25})
```

Compare that to writing this three times in driver-specific code every time you touch a different database. Flash eliminates that entirely.

Key design decisions

Decision	Why
No models or class definitions	Work with table names directly, like a query tool
No heavy ORM layer	Flash is a thin wrapper — you stay in control
Auto-commit by default	Inserts and updates commit instantly unless inside begin()
Built-in trigger hooks	Attach before/after callbacks to any operation, any table
Single unified filter dict	Write {"age": {">": 18}} once — Flash translates it

2. Installation

```
# Install Flash + the driver for the database you use
pip install pyflashdb[mysql]      # MySQL
pip install pyflashdb[postgres]  # PostgreSQL
pip install pyflashdb[mongodb]   # MongoDB
pip install pyflashdb[all]       # All three at once
```

What each extra installs

Extra	Driver	Min version
[mysql]	mysql-connector-python	8.0
[postgres]	psycopg2-binary	2.9
[mongodb]	pymongo	4.0

Verify installation

```
python -c "from flash import FlashDB; print('OK')"
```

3. Connecting to a Database

Constructor

```
FlashDB(db_type: str, config: dict)
```

db_type accepts: "mysql" · "postgres" / "postgresql" · "mongodb" / "mongo"

MySQL

```
from flash import FlashDB

flash = FlashDB("mysql", {
    "host":      "localhost",    # required
    "port":      3306,           # optional — default 3306
    "user":      "root",         # required
    "password":  "your_pass",    # required
    "database":  "mydb",         # required
})
```

PostgreSQL

```
flash = FlashDB("postgres", {
    "host":      "localhost",      # required
    "port":      5432,             # optional — default 5432
    "user":      "postgres",       # required
    "password":  "your_pass",      # required
    "database":  "mydb",           # required
})
```

MongoDB

```
# Local connection
flash = FlashDB("mongodb", {
    "host":      "localhost",      # optional — default localhost
    "port":      27017,           # optional — default 27017
    "database":  "mydb",          # required
})

# Atlas / authenticated — use uri
flash = FlashDB("mongodb", {
    "uri":       "mongodb+srv://user:pass@cluster.mongodb.net/",
    "database":  "mydb",
})
```

Context Manager (recommended)

Always prefer the context manager — the connection closes even if an exception occurs:

```
with FlashDB("mysql", config) as flash:
    flash.add("users", {"name": "Komal"})
    flash.update("users", {"name": "Komal"}, {"age": 26})
# connection closed automatically
```

Utilities

```
flash.ping()    # True if alive
flash.close()   # close the connection
```

4. CRUD — Insert

flash.add(table, data) — insert one record

```
flash.add(table: str, data: dict) -> int | str
```

Returns the auto-generated primary key. **int** for MySQL/PostgreSQL, **str** ObjectId for MongoDB.

```
uid = flash.add("users", {
    "name": "Komal",
    "email": "Komal@example.com",
    "age": 25,
    "score": 9.5,
})
print(f"Created ID: {uid}")
```

✓ Tip

Flash auto-commits after add(). You do not need to call commit() unless inside an explicit begin() block.

flash.bulk_insert(table, records) — insert many records

```
flash.bulk_insert(table: str, records: list[dict]) -> int
```

Returns the number of rows inserted. All records must have the same keys.

```
count = flash.bulk_insert("products", [
    {"name": "Widget A", "price": 9.99, "stock": 100},
    {"name": "Widget B", "price": 14.99, "stock": 250},
    {"name": "Widget C", "price": 4.99, "stock": 500},
])
print(f"Inserted {count} products")
```

5. CRUD — Read

flash.all(table) — every record

```
flash.all(table: str) -> list[dict]
```

```
for user in flash.all("users"):
    print(user["name"], user["age"])
```

■ Note

Do not call all() on large tables. Use select() with limit instead.

flash.select(table, ...) — full query

```
flash.select(
    table:      str,
    fields:     list[str] = None,    # columns to return; None = all
    filters:    dict       = None,   # filter dict (see Section 8)
    limit:      int        = None,   # max records
    offset:     int        = None,   # records to skip
    order_by:   str        = None,   # field; prefix "-" = desc (MongoDB)
) -> list[dict]
```

```
# All options at once
results = flash.select("users",
    fields    = ["name", "email", "age"],
    filters   = {"age": {">": 18}, "active": True},
    order_by  = "age",
    limit     = 20,
    offset    = 0,
)

# MongoDB newest-first
flash.select("posts", order_by="-created_at", limit=10)
```

flash.where(table, filters) — shorthand filter

```
flash.where(table: str, filters: dict, fields: list[str] = None) -> list[dict]
```

```
flash.where("orders", {"status": "pending"})
flash.where("users", {"age": {">": 18}})
flash.where("users", {"active": True}, fields=["name", "email"])
```

flash.find_one(table, filters) — first match

```
flash.find_one(table: str, filters: dict) -> dict | None
```

Returns the first match or `None`. Never raises on an empty result.

```
user = flash.find_one("users", {"email": "Komal@example.com"})
if user:
    print(user["name"])
```

flash.count(table, filters) — count without fetching

```
flash.count(table: str, filters: dict = None) -> int
```

```
total = flash.count("users")
adults = flash.count("users", {"age": {">": 18}})
```

flash.limit(table, n) — first N records

```
sample = flash.limit("events", 5)
```

flash.paginate(table, page, size) — page of results

```
flash.paginate(table: str, page: int = 1, size: int = 10) -> dict
```

Response shape:

```
{
  "data": [ {...}, {...} ], # records on this page
  "page": 2,                # current page (1-indexed)
  "size": 10,               # page size requested
  "total": 87,              # total records in table
}
```

```
page = flash.paginate("users", page=2, size=15)
print(f"Page {page['page']} — {len(page['data'])} of {page['total']} users")
```

6. CRUD — Update

flash.update(table, filters, data) — update matching records

```
flash.update(table: str, filters: dict, data: dict) -> int
```

Updates **all** records that match `filters`. Returns the number modified.

```
# One field
flash.update("users", {"name": "Komal"}, {"age": 26})

# Multiple fields
flash.update("products", {"id": 42}, {
    "price": 19.99,
    "stock": 0,
    "active": False,
})

# All matching records
updated = flash.update("orders", {"status": "processing"}, {"status": "shipped"})
print(f"Shipped {updated} orders")
```

Note

update() modifies every matching row. To update exactly one record, filter by a unique field such as id.

7. CRUD — Delete

flash.delete(table, filters) — delete matching records

```
flash.delete(table: str, filters: dict = None) -> int
```

Returns the number of records deleted.

```
flash.delete("users", {"name": "Komal"})
flash.delete("sessions", {"expires_at": {"<": "2024-01-01"}})
```

Danger

Danger: flash.delete("users") with no filter deletes EVERY record. Always pass a filter.

8. Filter Syntax

Flash uses a single filter dictionary. It automatically translates to SQL WHERE clauses or MongoDB query dicts.

Operator table

Flash operator	SQL produced	MongoDB produced	Example
>	field > %s	\$gt	{"age": {">": 18}}

<	field < %s	\$lt	{"age": {"<": 65}}
>=	field >= %s	\$gte	{"score": {">=": 80}}
<=	field <= %s	\$lte	{"score": {"<=": 100}}
!=	field != %s	\$ne	{"role": {"!=": "guest"}}
(plain value)	field = %s	direct equality	{"name": "Komal"}
in	field IN (...)	\$in	{"role": {"in": ["admin", "mod"]}}
not in	field NOT IN (...)	\$nin	{"status": {"not in": ["banned"]}}
like	field LIKE %s	\$regex (converted)	{"name": {"like": "Jo%"}}

Filter examples

```
# Exact match — no operator needed
flash.where("users", {"name": "Komal"})
flash.where("users", {"active": True})

# Comparison
flash.where("users", {"age": {">": 18}})
flash.where("orders", {"total": {">=": 100}})

# Range — multiple operators on one field
flash.where("users", {"age": {">=": 18, "<=": 65}})
flash.where("products", {"price": {">": 5, "<": 50}})

# IN / NOT IN
flash.where("users", {"role": {"in": ["admin", "moderator"]}})
flash.where("users", {"status": {"not in": ["banned", "suspended"]}})

# LIKE / regex
flash.where("users", {"email": {"like": "%@gmail.com"}})

# Multiple fields — all joined with AND
flash.where("users", {
    "age": {">": 18},
    "active": True,
    "role": {"in": ["user", "admin"]},
})
```

like on MongoDB

Flash automatically converts SQL LIKE wildcards to regex before sending to MongoDB:

SQL LIKE pattern	Regex sent to MongoDB
------------------	-----------------------

Jo%	Jo.*
J_hn	J.hn
%@gmail.com	.*@gmail.com

You write SQL-style patterns everywhere. Flash handles the conversion.

9. Schema Operations

`flash.create_table(table, schema, primary_key="id")`

Creates a table (SQL) or collection with optional JSON Schema validation (MongoDB). Safe to call repeatedly — uses CREATE TABLE IF NOT EXISTS.

```
flash.create_table("users", {
    "id":          "int",          # becomes AUTO_INCREMENT / SERIAL primary key
    "name":        "str",
    "email":       "str",
    "bio":         "text",
    "score":       "float",
    "is_active":   "bool",
    "created_at":  "datetime",
    "metadata":    "json",
}, primary_key="id")
```

Type mapping

Flash type	MySQL	PostgreSQL	MongoDB BSON
int / integer	INT	INTEGER	int
str / string	VARCHAR(255)	VARCHAR(255)	string
text	TEXT	TEXT	string
float	FLOAT	REAL	double
double	DOUBLE	DOUBLE PRECISION	double
bool / boolean	TINYINT(1)	BOOLEAN	bool
date	DATE	DATE	date
datetime	DATETIME	TIMESTAMP	date

timestamp	TIMESTAMP	TIMESTAMP	date
json	JSON	JSONB	object
blob	BLOB	BYTEA	—

Other schema methods

```
flash.drop_table("users")    # permanently drop table / collection
flash.truncate("users")     # delete all rows, keep structure
flash.show_tables()         # -> ["users", "orders", ...]
flash.describe("users")     # -> list of column-definition dicts
```

10. Trigger Hooks

Flash fires Python callables before or after any database operation. This works on all three databases — including MongoDB, which has no native trigger support.

How they work

- Hooks are **per-table** and **per-instance** — hooks registered on one FlashDB instance never affect another
- **Before hooks** receive the data payload going into the database
- **After hooks** receive the payload plus the result (inserted ID, row count, etc.)
- Multiple hooks can be attached to the same event — all run in registration order
- Exceptions inside a hook are caught and printed as warnings; they never abort the database operation

Hook reference

Decorator	When it fires	Before signature	After signature
<code>before_insert(table)</code>	<code>add()</code> / <code>bulk_insert()</code>	<code>fn(data)</code>	—
<code>after_insert(table)</code>	<code>add()</code> / <code>bulk_insert()</code>	—	<code>fn(data, result)</code>
<code>before_update(table)</code>	<code>update()</code>	<code>fn(payload)</code>	—
<code>after_update(table)</code>	<code>update()</code>	—	<code>fn(payload, count)</code>
<code>before_delete(table)</code>	<code>delete()</code>	<code>fn(filters)</code>	—
<code>after_delete(table)</code>	<code>delete()</code>	—	<code>fn(filters, count)</code>
<code>before_select(table)</code>	<code>all()</code> / <code>select()</code> / <code>where()</code>	<code>fn(payload)</code>	—
<code>after_select(table)</code>	any read	—	<code>fn(payload, results)</code>

For update hooks: `payload = {"filters": ..., "data": ...}`

Decorator style

```

@flash.before_insert("users")
def validate(data):
    if not data.get("email"):
        raise ValueError("Email is required")
    if "@" not in data.get("email", ""):
        raise ValueError("Invalid email format")

@flash.after_insert("users")
def on_created(data, result):
    print(f"[AUDIT] Created user '{data['name']}' -> ID {result}")

@flash.before_update("users")
def log_update(payload):
    print(f"[LOG] Update {payload['filters']} -> {payload['data']}")

@flash.after_delete("orders")
def on_deleted(filters, count):
    print(f"[ALERT] Deleted {count} order(s) matching {filters}")

```

Programmatic style

```

def audit(data):
    print(f"Insert: {data}")

flash.add_hook("before", "insert", "payments", audit)
# add_hook(timing, event, table, fn)
# timing: "before" | "after"
# event:  "insert" | "update" | "delete" | "select"

```

Managing hooks

```

flash.list_hooks()           # -> dict summary of all registered hooks
flash.clear_hooks()          # remove all hooks on all tables
flash.clear_hooks("users")   # remove hooks for one table only

```

11. Transactions

Group multiple operations atomically — either all succeed or all roll back.

Method	Description
flash.begin()	Start a transaction. Disables auto-commit.
flash.commit()	Persist all changes since begin().
flash.rollback()	Undo all changes since begin().

Note

MySQL / PostgreSQL: Full support. MongoDB: Requires a replica set (MongoDB 4.0+). Not available on standalone instances.

Bank transfer example

```
flash.begin()
try:
    flash.update("accounts", {"user_id": 1}, {"balance": 400}) # debit
    flash.update("accounts", {"user_id": 2}, {"balance": 600}) # credit
    flash.add("transfer_log", {"from": 1, "to": 2, "amount": 100})
    flash.commit()
    print("Transfer complete")
except Exception as e:
    flash.rollback()
    print(f"Rolled back: {e}")
```

Note

Inside begin(), Flash does NOT auto-commit individual operations. You must call commit() to persist.

Order processing example

```
flash.begin()
try:
    order_id = flash.add("orders", {"user_id": 7, "total": 149.99, "status": "pending"})
    flash.bulk_insert("order_items", [
        {"order_id": order_id, "product_id": 1, "qty": 2},
        {"order_id": order_id, "product_id": 5, "qty": 1},
    ])
    flash.update("inventory", {"product_id": 1}, {"stock": 48})
    flash.update("inventory", {"product_id": 5}, {"stock": 19})
    flash.commit()
except Exception as e:
    flash.rollback()
    print(f"Order failed: {e}")
```

12. Raw Queries

Escape hatch for anything Flash's API cannot express natively.

SQL

```
flash.raw(sql: str, params: list = None) -> list[dict] | int
```

```
# Simple read
results = flash.raw("SELECT * FROM users WHERE age > 18")

# Parameterised — always use %s (prevents SQL injection)
results = flash.raw(
    "SELECT name, email FROM users WHERE age > %s AND active = %s",
    [18, True]
)

# JOIN
results = flash.raw("""
    SELECT u.name, o.total
    FROM users u
    JOIN orders o ON o.user_id = u.id
    WHERE o.status = %s
""", ["pending"])

# Non-SELECT -> returns affected row count
count = flash.raw(
    "UPDATE users SET verified = 1 WHERE email LIKE %s",
    ["%@company.com"]
)
```

MongoDB

```
flash.raw(command, table: str = None) -> list | dict
```

```
# Collection-level find with native operators
results = flash.raw({"age": {"$gt": 18, "$lt": 30}}, table="users")

# Database-level command
stats = flash.raw("dbStats")
```

13. MongoDB-Specific Features

flash.aggregate(table, pipeline)

```
flash.aggregate(table: str, pipeline: list) -> list[dict]
```

```
# Revenue by category
flash.aggregate("orders", [
    {"$match": {"status": "paid"}},
    {"$unwind": "$items"},
    {"$group": {"_id": "$items.category", "revenue": {"$sum": "$items.price"}}},
    {"$sort": {"revenue": -1}},
    {"$limit": 10},
])

# Average age per role
flash.aggregate("users", [
    {"$group": {"_id": "$role", "avg_age": {"$avg": "$age"}}},
    {"$sort": {"avg_age": -1}},
])
```

flash.create_index(table, field, unique=False)

```
flash.create_index("users", "email", unique=True) # enforce unique
flash.create_index("orders", "user_id")          # speed up lookups
flash.create_index("products", "category")
```

14. Error Handling

Exception hierarchy

```
FlashError                <- base class; catch this for any Flash error
+--> FlashConnectionError  <- connection failed
+--> QueryError            <- query execution failed
+--> TransactionError      <- commit / rollback failed
+--> SchemaError           <- create / drop / truncate failed
+--> UnsupportedDatabaseError <- unknown db_type passed to FlashDB()
```

All exceptions are importable directly from flash:

```

from flash import (
    FlashError,
    FlashConnectionError,
    QueryError,
    TransactionError,
    SchemaError,
    UnsupportedDatabaseError,
)

```

Usage

```

from flash import FlashDB, FlashConnectionError, QueryError

try:
    flash = FlashDB("mysql", config)
except FlashConnectionError as e:
    print(f"Cannot connect: {e}")
    raise SystemExit(1)

try:
    flash.add("users", bad_data)
except QueryError as e:
    print(f"Insert failed: {e}")

```

Common errors

Error message	Cause	Fix
MySQL connection failed: 2003	MySQL not running or wrong host/port	Start MySQL; check host and port
MySQL connection failed: 1045	Wrong credentials	Check user and password
MySQL connection failed: 1049	DB does not exist	CREATE DATABASE mydb;
psycopg2 is not installed	Missing driver	pip install psycopg2-binary
mysql-connector not installed	Missing driver	pip install mysql-connector-python
pymongo is not installed	Missing driver	pip install pymongo
UnsupportedDatabaseError	Unknown db_type	Use "mysql", "postgres", or "mongodb"
Table 'x' doesn't exist	Table not created	Call flash.create_table(...) first
TransactionError: Commit failed	commit() without begin()	Call flash.begin() first

15. Verifying Your Setup

Step 1 — Check the server is running

```
# MySQL
mysqladmin -u root -p status

# PostgreSQL
pg_isready

# MongoDB
mongosh --eval "db.runCommand({ping: 1})"
```

Step 2 — Create a test database

```
# MySQL
mysql -u root -p -e "CREATE DATABASE flash_test;"

# PostgreSQL
psql -U postgres -c "CREATE DATABASE flash_test;"

# MongoDB — nothing needed, Flash creates it on first insert
```

Step 3 — Run the verification script

Save as `verify.py`, edit the three config lines at the top, then run `python verify.py`.

```

from flash import FlashDB, FlashConnectionError

# -- Edit these --
DB_TYPE = "mysql"
DB_NAME = "flash_test"
CONFIG = {

    "host": "localhost",
    "user": "root",
    "password": "your_password",
    "database": DB_NAME,
}

print(f"\n pyflashdb v1.0.4 -- {DB_TYPE.upper()} verification\n")

try:
    flash = FlashDB(DB_TYPE, CONFIG)
    print("Connected")
except FlashConnectionError as e:
    print(f"Error: {e}"); raise SystemExit(1)

assert flash.ping()
T = "flash_verify"
flash.create_table(T, {"id":"int","name":"str","email":"str","age":"int","active":"bool"})
uid = flash.add(T, {"name":"Komal","email":"k@t.com","age":21,"active":True})

flash.bulk_insert(T, [
    {"name":"Aman", "email":"am@t.com", "age":17, "active":True},
    {"name":"Mohit", "email":"mo@t.com", "age":30, "active":True},
    {"name":"Shree", "email":"sh@t.com", "age":30, "active":True},
    {"name":"Pranali Veer", "email":"pv@t.com", "age":30, "active":True},
    {"name":"Kartiki", "email":"ka@t.com", "age":30, "active":True},
    {"name":"Pranali Dhore", "email":"pd@t.com", "age":30, "active":True},
    {"name":"Disha", "email":"di@t.com", "age":30, "active":True},
    {"name":"Ronit", "email":"ro@t.com", "age":30, "active":True},
    {"name":"Shravani", "email":"sv@t.com", "age":30, "active":True},
    {"name":"Ritesh", "email":"ri@t.com", "age":30, "active":True},
])

assert len(flash.all(T)) == 11 # 1 add + 10 bulk
assert len(flash.where(T, {"age":{">":18}})) == 10 # everyone except Aman
assert flash.update(T, {"name":"Komal"}, {"age":22}) == 1
assert flash.delete(T, {"name":"Aman"}) == 1

flash.truncate(T); flash.drop_table(T); flash.close()

print("All checks passed -- pyflashdb is working!")

```

Expected output

```
pyflashdb v1.0.4 -- MySQL verification
Connected
All checks passed -- pyflashdb is working!
```

16. Feature Matrix

Feature	MySQL	PostgreSQL	MongoDB
all()	Yes	Yes	Yes
select()	Yes	Yes	Yes
where()	Yes	Yes	Yes
find_one()	Yes	Yes	Yes
count()	Yes	Yes	Yes
limit()	Yes	Yes	Yes
paginate()	Yes	Yes	Yes
add()	Yes	Yes	Yes
bulk_insert()	Yes	Yes	Yes
update()	Yes	Yes	Yes
delete()	Yes	Yes	Yes
create_table()	Yes	Yes	Yes (JSON Schema)
drop_table()	Yes	Yes	Yes
truncate()	Yes	Yes	Yes
show_tables()	Yes	Yes	Yes
describe()	Yes	Yes	collStats only
begin/commit/rollback	Yes	Yes	Replica set required
raw()	SQL string	SQL string	Command dict
aggregate()	No	No	Yes
create_index()	No	No	Yes
Trigger hooks	Yes	Yes	Yes (emulated)
Auto-commit	Yes	Yes	N/A

17. Project Structure

```

pyflashdb/
+--> flash/
|   +--> __init__.py           <- FlashDB + exceptions; version = "1.0.4"
|   +--> core.py               <- FlashDB class - the entire public API
|   +--> filters.py            <- filter dict -> SQL WHERE / MongoDB query
|   +--> triggers.py           <- hook registry + decorator API
|   +--> exceptions.py         <- FlashError hierarchy
|   +--> adapters/
|       +--> __init__.py
|       +--> mysql_adapter.py   <- MySQL via mysql-connector-python
|       +--> postgres_adapter.py <- PostgreSQL via psycopg2
|       +--> mongo_adapter.py   <- MongoDB via pymongo
+--> setup.py
+--> pyproject.toml
+--> MANIFEST.in
+--> LICENSE
+--> CHANGELOG.md
+--> README.md

```

How a call flows through the library

```

flash.add("users", data)
|
+--> before_insert triggers fire      (flash/triggers.py)
|
+--> adapter.add(table, data)          (flash/adapters/mysql_adapter.py)
|   builds SQL / pymongo call
|   executes against the DB
|   returns inserted ID
|
+--> after_insert triggers fire
|
+--> returns inserted ID to caller

```

Publishing to PyPI

```

pip install build twine
python -m build
twine upload dist/*

```

18. Changelog

v1.0.4 — Bug Fix Release

- **FlashConnectionError rename** — `ConnectionError` renamed to `FlashConnectionError`. The old name silently shadowed Python's built-in `ConnectionError`. A private backwards-compatibility alias is kept inside `flash.exceptions` only.
- **PostgreSQL `add()` return type** — now returns an `int` (the inserted row ID) consistent with MySQL. Previously returned the entire row as a dict due to `RETURNING *`.
- **like filter on MongoDB** — SQL `LIKE` wildcards (`%`, `_`) are now correctly converted to regex (`.`, `.`), and regex meta-characters in the pattern are escaped.
- **`__version__` corrected** — updated from `"1.0.0"` to `"1.0.4"`.
- **All exceptions exported** — `TransactionError`, `SchemaError`, and `UnsupportedDatabaseError` are now importable directly from `flash`.
- **`raw()` routing fixed** — MongoDB's `table` argument is now correctly forwarded. SQL and MongoDB adapters each receive exactly the arguments they expect.

v1.0.3

Initial public PyPI release.

pyflashdb v1.0.4 · MIT License · Python 3.8+ · pypi.org/project/pyflashdb/